

New Models and Services

o3-mini <https://openai.com/index/openai-o3-mini/> <https://cdn.openai.com/o3-mini-system-card.pdf>

This is a new reasoning model by OpenAI, which costs like o1-mini but performs on par with o1. This model has a new **reasoning_effort** parameter controlling the reasoning length. It has three potential values: “low”, “medium”, and “high” (default is “medium”).

Here are some of the results. The shaded areas show the improvement in quality from taking the majority vote of 64 LLM calls. (That should be costly!) It seems, that o3 mini is a good upgrade in quality. (Though Anthropic Claude Sonnet 3.5 seems to be still better in coding.)



An interesting thing mentioned in the model card is **Deliberate Alignment** as a tool for establishing alignment (safety etc). Usual ways of doing this (RLHF and DPO) would involve training an LLM to optimize numerical reward (RLHF) or to discern between accepted and rejected answers from a pair (DPO). In both cases, the LLM needs to learn from a signal which in a way is implicit for it. With a reasoning model, alignment may be introduced into the reasoning itself in the training data. For that, you may do the following:

- First, add a special system prompt enforcing some type of alignment and generate outputs with it from different prompts,
- As training stage, remove the system prompt. This way, the LLM will learn to exhibit specific “safe” thinking patterns even when not being explicitly prompted for it.

Deep Research by OpenAI <https://openai.com/index/introducing-deep-research/>

We've been enjoying this service with Google's Gemini for a while, and now it's time for OpenAI to introduce their version. That's how it works: you provide a prompt, and a version of o3, optimized for web search, compiles a report by analyzing and synthesizing various online sources.

For now, it's only available for Pro users (and with a cap of 100 queries / month), but Plus tier will also get access at some point (even more strictly capped, of course). I haven't tried it yet, but those who tried tend to be happy with it. Deep Research also shows promising evaluation results. For example, it gets a nice improvement on the [Humanity's Last Exam](#), which is a very tough and broad academic benchmark gathered by researchers in hope to build a lasting evaluation tool against the rapid development of AI:

Model	Accuracy (%)
GPT-4o	3.3
Grok-2	3.8
Claude 3.5 Sonnet	4.3
Gemini Thinking	6.2
OpenAI o1	9.1
DeepSeek-R1*	9.4
OpenAI o3-mini (medium)*	10.5
OpenAI o3-mini (high)*	13.0
OpenAI deep research**	26.6

* Model is not multi-modal, evaluated on text-only subset.
**with browsing + python tools

Open source DeepResearch by Hugging Face, made in just 24h :D

<https://huggingface.co/blog/open-deep-research>

There's an interesting insight about tool usage here. Two tools were needed for the task – a web browser and a text inspector – and Hugging Face tried two approaches to implementing them:

- Agentic tool-usage approach with JSON-generation
- LLM writing actions in code!

The second approach led to 33% -> 55.15% improvement on the GAIA benchmark.

That's very exciting, I think.

s1: Simple test-time scaling <https://arxiv.org/pdf/2501.19393>

In short, the authors of this paper decided to take Qwen2.5-32B-Instruct and find a cheap way of turning it into a highly capable reasoning model.

“Test-time scaling” refers to various attempts at getting better quality through spending more time and money on generation. There are two basic approaches:

- **Parallel** approach is just asking the same question many times with majority voting in the end,
- **Sequential** approach would traditionally involve iterative self-correction and rewriting, but recently became more about long, non-linear reasoning. The latter is in a sense also iterative improvement, but generated naturally, not orchestrated with sequential prompts.

There are hybrid strategies as well. You can find interesting treatment of pre-long-reasoning-era hybrid strategies and the trade-off between parallel and sequential scaling in [this paper](#).

The authors of s1, inspired by the progress made by o1 and DeepSeek, decided to leverage long thinking. And for that, they employed two ideas:

- Selecting 1000 high-quality reasoning examples for fine tuning.
 - First, they collect 59,029 questions from 16 diverse sources (including many Olympiad problems).
 - For each question, they generate a reasoning trace and solution using the experimental Google Gemini Flash Thinking API.
 - Then, they filter the problems, first by Quality (no poor formatting etc), then by Difficulty. A problem is deemed difficult if neither Qwen2.5-7B-Instruct nor Qwen2.5-32B-Instruct is able to solve it, and also the reasoning length is large enough.
 - Finally, they sample 1000 problems in a stratified way with respect to a number of topics.
- Introducing **budget forcing**. This is a curious idea which is aligned with the observations that the true power beyond reasoning is the sheer amount of compute that allows the right things to happen deep inside LLM layers (See [Let's Think Dot by Dot](#) paper for a discussion of this; see also [Quiet-STaR](#) and the [Reasoning in the Continuous Latent Space](#) paper).

The authors suggest making an LLM to reason for a longer time by just introducing “Wait” every time it’s going to produce an answer, until some target length is hit. “Wait” prompts the LLM to double-check everything or maybe explore a new approach.

As a result, the model they obtain seem to scale up with increasing budget better than it happens under Self-Consistency (which was actually considered a very effective approach for extra-high budgets).

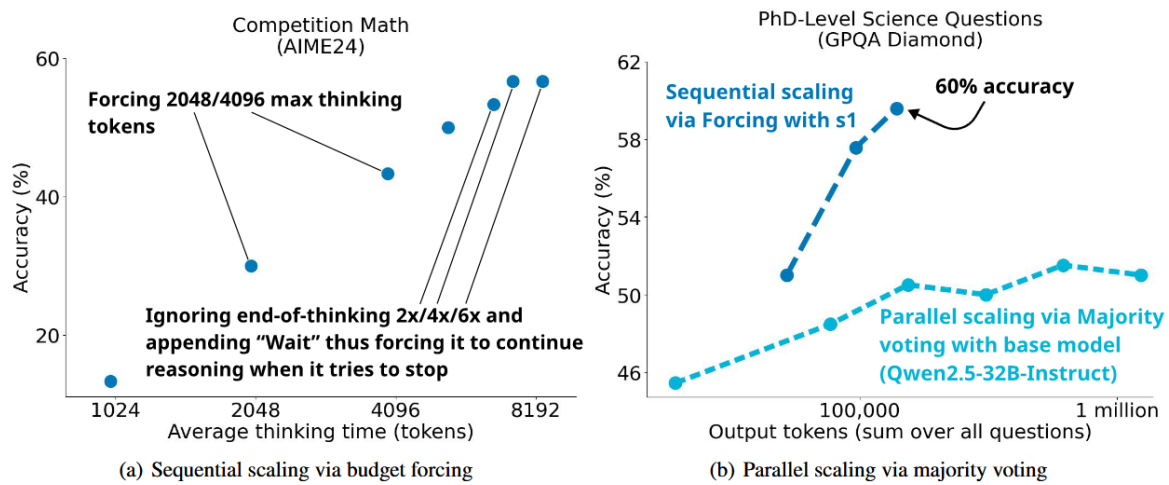


Figure 4. Sequential and parallel test-time scaling. (a): Budget forcing shows clear scaling trends and extrapolates to some extent. For the three rightmost dots, we prevent the model from stopping its thinking 2/4/6 times, each time appending “Wait” to its current reasoning trace. (b): For Qwen2.5-32B-Instruct we perform 64 evaluations for each sample and visualize the performance when majority voting across 2, 4, 8, 16, 32, and 64 of these.

They also report quite competitive benchmark scores:

Reasoning performance		Budget forcing		
Model	# ex.	AIME 2024	MATH-500	GPQA Diamond
API only				
o1-preview	N.A.	44.6	85.5	73.3
o1-mini	N.A.	70.0	90.0	60.0
o1	N.A.	74.4	94.8	77.3
Gemini 2.0 Flash Think.	N.A.	60.0	N.A.	N.A.
Open Weights				
Qwen2.5-32B-Instruct	N.A.	26.7	84.0	49.0
QwQ-32B	N.A.	50.0	90.6	65.2
r1	≫800K	79.8	97.3	71.5
r1-distill	800K	72.6	94.3	62.1
Open Weights and Open Data				
Sky-T1	17K	43.3	82.4	56.8
Bespoke-32B	17K	63.3	93.0	58.1
s1 w/o BF	1K	50.0	92.6	56.6
s1-32B	1K	56.7	93.0	59.6

Thoughts Are All Over the Place: On the Underthinking of o1-Like LLMs <https://arxiv.org/pdf/2501.18585>

This paper shares interesting insights about what happens when reasoning models make mistakes.

For an LLM's answer, you can study its length (the number of tokens), but also the **number of "thoughts"** and the average thought length. "Thoughts" here are individual reasoning chains, divided by words such as "Wait" or "Alternatively". Non surprisingly, there is a correlation between the question difficulty and the answer's length, both in terms of tokens and thoughts:

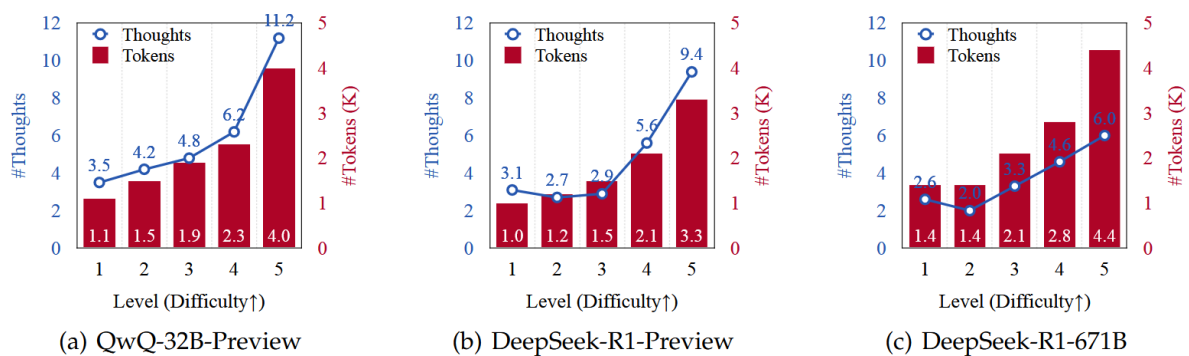


Figure 3: Average number of thoughts ("Thoughts") and tokens ("Tokens") in generated responses across different difficulty levels of the MATH500 test set.

What's more interesting is that when reasoning LLMs make mistakes, they switch thoughts more frequently.

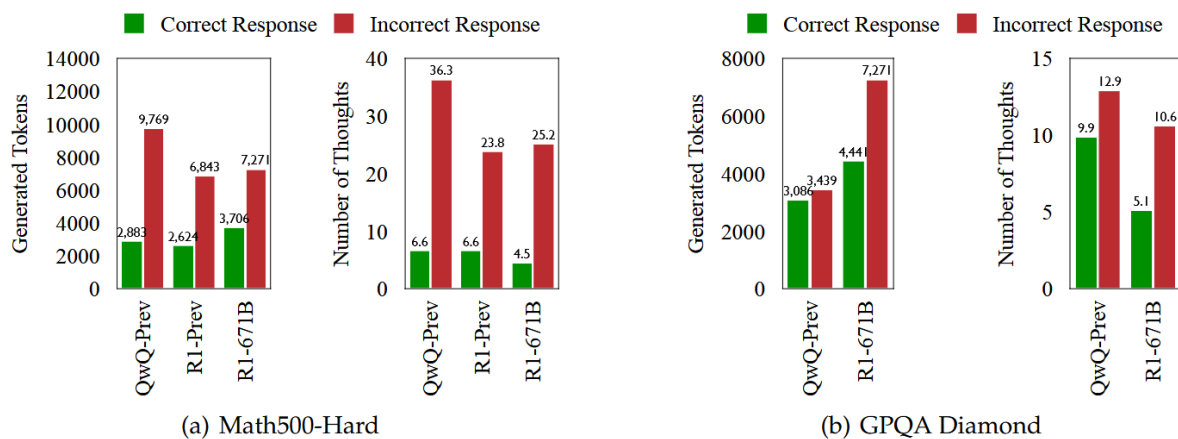


Figure 4: O1-like LLMs switch thinking more frequently on incorrect responses, thus expend more tokens without contributing to accuracy.

This is not clearly visible from the bar plots above, so I've made a table for Math500-Hard:

	CORRECT			INCORRECT		
	n_tokens	n_thoughts	token_per_thought	n_tokens	n_thoughts	token_per_thought
QwQ-prev	2.883	6.6	436.82	9.769	36.3	269.12
R1-prev	2.624	6.6	397.58	6.843	23.8	287.52
R1	3.706	4.5	823.56	7.271	25.2	288.53

And here, indeed, we see that in incorrect answers we have less tokens per thought. And that’s not just a coincidence. Using LLM as a Judge, the authors measure the correctness of thoughts (whether they could lead to the correct answer, if completed). It turns out that in incorrect solutions promising thoughts are oftentimes prematurely abandoned. The author nickname this phenomenon **underthinking**.

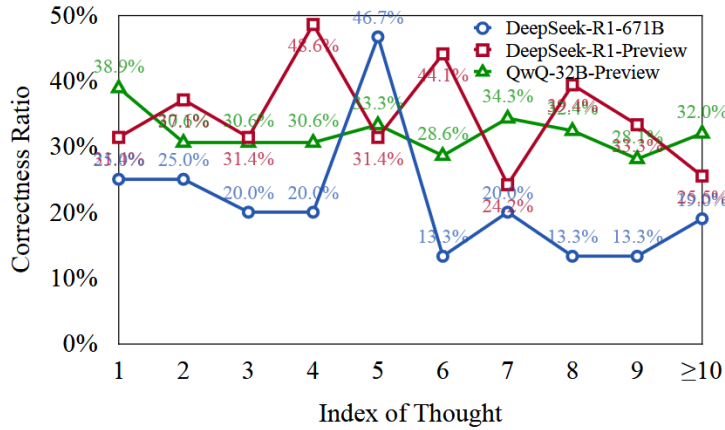


Figure 5: The ratio of correct reasoning thoughts at each index in incorrect responses. A notable portion of early-stage thoughts (e.g., the first few thoughts) are correct but abandoned without being fully explored.

The authors also introduce the **underthinking metric** to evaluate how much different LLMs are susceptible to this. For a query-incorrect response pair, it’s defined as

$$1 - \frac{\hat{T}_i}{T_i},$$

where

- T_i is the total number of tokens the response,
- $\hat{T}_i$ is the number of tokens from the beginning of that response up to and including the first correct though (that is thought which would lead to the correct answer if LLM hadn’t abandoned it).

Table 1: Underthinking scores on challenging testsets

Models	Accuracy	UT Score
<i>MATH500-Hard (Level 5)</i>		
QwQ-32B-Preview	84.3	58.2
DeepSeek-R1-Preview	83.6	61.5
DeepSeek-R1-671B	92.5	65.4
<i>GPQA Diamond</i>		
QwQ-32B-Preview	59.6	48.3
DeepSeek-R1-671B	73.2	58.8
<i>AIME24</i>		
QwQ-32B-Preview	46.7	65.0
DeepSeek-R1-Preview	46.7	75.7
DeepSeek-R1-671B	73.3	37.0

They calculate this metric for several LLMs on different datasets, and they get pretty impressive average underthinking scores.

To address this problem, the authors suggest **Thought Switching Penalty (TIP)**. The idea is to introduce an additional transformation of logits before softmax: penalizing logits of thought changing tokens (“alternatively”, “wait” etc) for a certain duration after the start of each thought.

With this, they are able to lower the models’ underthinking score a bit, which is also beneficial for accuracy.

Table 3: Results of the proposed decoding with TIP.

Models	Pass@1	
	Accuracy(↑)	UT Score (↓)
<i>MATH500-Hard (Level 5)</i>		
QwQ-32B-Preview	82.8	71.1
+ TIP	84.3	69.7
<i>GPQA Diamond</i>		
QwQ-32B-Preview	57.1	59.1
+ TIP	59.3	56.5
<i>AIME2024</i>		
QwQ-32B-Preview	41.7	72.4
+ TIP	45.8	68.2

Large Language Models Think Too Fast To Explore Effectively <https://arxiv.org/pdf/2501.18009>

This is yet another study of LLM shortcomings. The authors study LLMs’ exploration ability or lack thereof. (And this seems to be well connected with LLM long reasoning capabilities again!)

The authors experiment with the [Little Alchemy 2](#) game (try it!), where you start with Air, Water, Earth, and Fire, and combine pairs of elements to get new elements. For example, Air + Earth = Dust and Dust + Fire = Gunpowder. Not all pairs result in a new element, so you need to be patient and inquisitive (and grasp the logic behind the game) to excel in that. You can check the results. An average human beats almost any LLM except from o1 which surpasses human proficiency at a large margin.

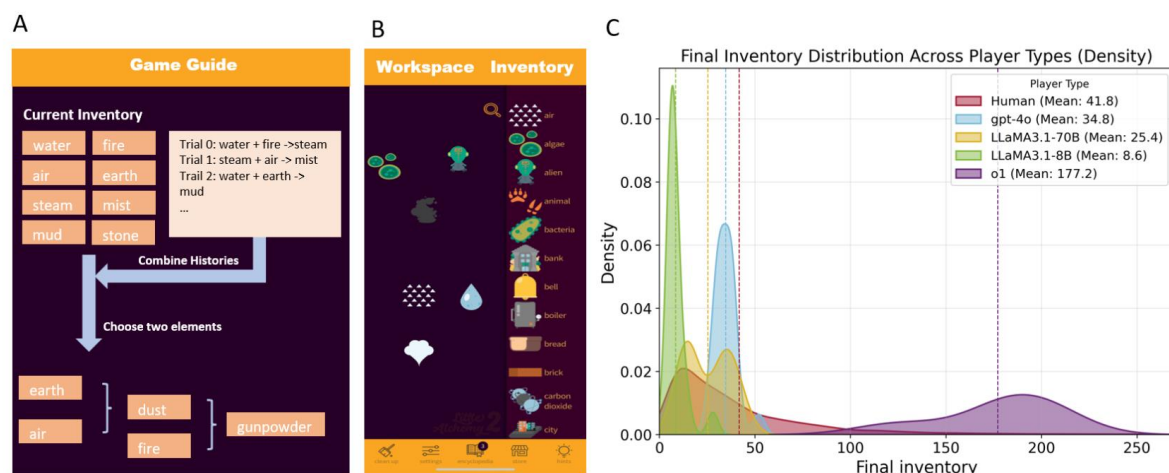


Figure 1: **A: LLMs Game Process.** LLMs select two elements per trial based on the inventory and trial history. **B: Human Game Interface.** Players select two elements to discover new elements, added to the inventory. **C: LLMs and Human Performance.**

Of course, LLMs don't have a fancy interface, so they rely on prompts

A: LLMs Game Original Prompt

Welcome to the Alchemy Game! You start with four basic elements: water, fire, earth, and air.

The objective of the game is to combine elements to create new ones. Each successful combination adds a new element to your inventory, which can be used for future combinations. Choose two elements to combine by writing them in the format 'element + element'. You can choose the same element twice or two different elements, and the order of the elements does not matter. Each combination produces deterministic.

Only output the combination, no other words. And you need try your best to create more elements. Let's get started!

Current Inventory:
Past Attempts:
The output format should be 'element1 + element2'. Only output the element combination, no other words. Give me one combination every time.

B: LLMs Game Prompt Engineering

Welcome to the Alchemy Game! You start with four basic elements: water, fire, earth, and air.

The objective of the game is to combine elements to create new ones. Each successful combination adds a new element to your inventory, which can be used for future combinations. You can choose the same element twice or two different elements, and the order of the elements does not matter.

Each combination produces deterministic results, meaning there is only one fixed outcome for each combination. Results cannot change upon repetition. Do not hypothesize new outcomes for previously combinations.

Step 1: Analyze past attempts to identify patterns (e.g., which elements combine well).
Step 2: Choose a new combination based on patterns and unused pairs.

And you need try your best to create more elements. Let's get started!

Current Inventory:
Past Attempts:
Before finalizing, check:
- Has this combination been tried before?
- If yes, choose a new one.
- If no, proceed.

The output format should be 'element1 + element2'. Only output the element combination, no other words. Give me one combination every time.

While studying LLM exploration patterns, the authors try to discern between two strategies: empowerment and uncertainty, leveraging the approach of [this Nature paper](#).

- **Empowerment** relates to searching for elements that promise to further open many new elements for us. An element is the more “empowering” the more distinct elements it can lead to by combination with all other possible elements in the game, provided that the true game tree is known.

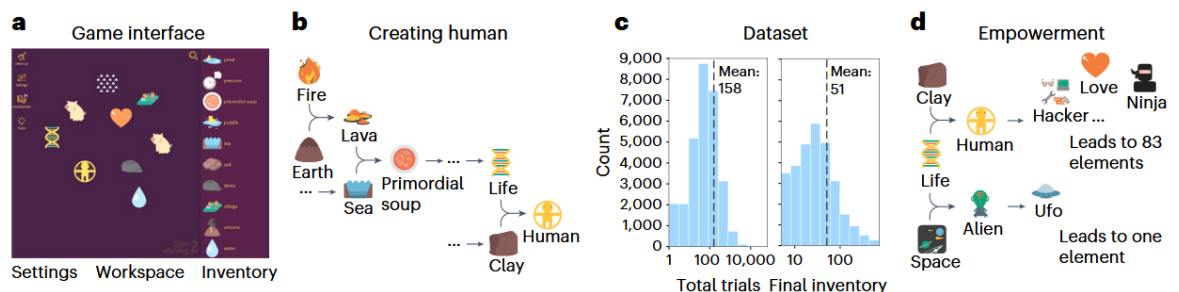


Fig. 1 | 'Little Alchemy 2'. **a**, Game interface. Players can use the workspace to combine two elements in an attempt to create new elements, which get added to the inventory and can be used for future combinations. **b**, Creating human. 'Little Alchemy 2' is a richly structured game in which semantically meaningful combinations can create up to 720 unique elements. **c**, Dataset of 29,493 players attempting a total of 4,691,033 combinations. Participants played for 158 trials

and found 51 elements on average. **d**, Example of empowerment algorithm. Combining life with clay creates human, which can lead to 83 other elements in total, such as hacker, love and ninja. Combining life with space creates alien, which can only lead to one other element, UFO. Thus, combining life with clay is the more empowering action.

(screenshot from the Nature paper)

In a sense, empowerment strategy is focused on long-term goals.

Technically, the estimation of an element's empowerment is somewhat tricky, but there is a clear idea behind updating it in-game. Namely, when we successfully combine two elements (and get a new one), we slightly increase the empowerment of both elements, and when the combination gives nothing, we slightly decrease them.

- **Uncertainty** refers to our lack of knowledge about a certain element: the less we tried it, the more its uncertainty. The uncertainty of an element e can be estimated as

$$\sqrt{\frac{\log T}{t_e + 1}},$$

where T is the total number of trials and t_e is the number of trials that used the element e . In a sense, uncertainty-driven strategies focus on short-term goals.

To understand how much both strategies contribute to overall LLM's exploration, the authors do the following. They sample a balanced multiclass classification dataset (x_i, y_i) , where

$x_i = (n_step, empowerment(e), uncertainty(e))$,

$y_i = (e \text{ is chosen/not chosen at step number } n_step)$.

Then they analyze the regression coefficients at empowerment and uncertainty trying to understand whether these two entities are important for the LLM. It turns out that, except for o1, LLMs only use uncertainty-driven exploration, which can probably explain their shortcomings. In turn, humans are more driven by empowerment in this game than by uncertainty.

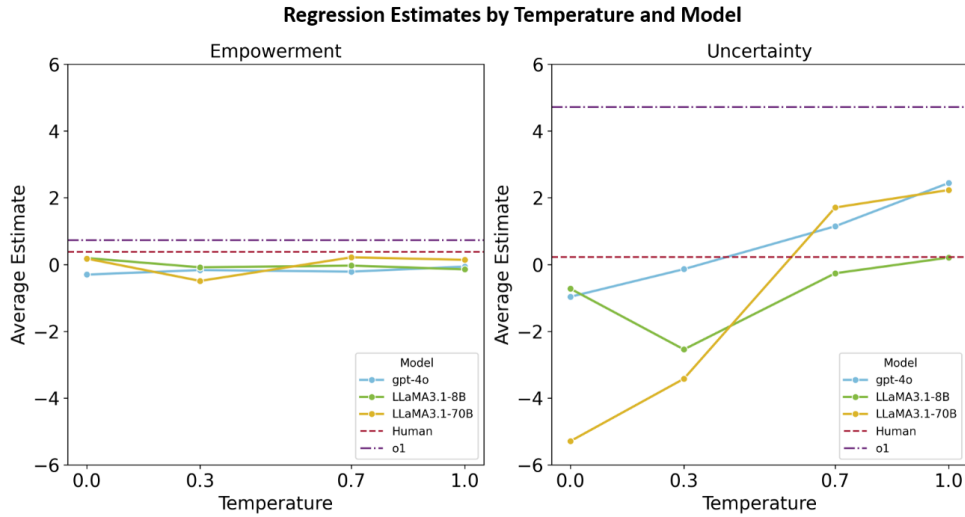


Figure 3: Regression Estimates by Temperature and Model. All models show lower empowerment weights than humans, except o1. As temperature increases, uncertainty weights rise, with o1 showing the highest weights across all models and humans.

Diverse Preference Optimization

<https://arxiv.org/pdf/2501.18101>

If you have ever used ChatGPT or any other top-tier LLM for creative tasks, you know how annoyingly repetitive their writings are in terms of both content and style. For example, when I asked Llama-3.1-8B to generate a poem about the raven several times with various temperatures up to 1, the first two words were the same every time :O (I'm even not telling you that the meter was the same as well.)

I like calling this “mode collapsing“, comparing this to what often happened with GANs. And I was excited to see a paper trying to address this issue.

I've heard more than once that SFT and RLHF/DPO are to blame for mode collapsing, and this paper cites a number of research efforts pointing at RLHF as a diversity reducing device. You can get a feeling of this if you recall what the RLHF objective looks like:

$$\mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(x)} r(x, y) - \beta KL(\pi_{\theta} \parallel \pi_{ref}) = \mathbb{E} \left(r(x, y) - \beta \frac{\pi_{\theta}(y|x)}{\pi_{ref}(y|x)} \right)$$

Optimizing it is a trade-off between optimizing the reward $r(x, y)$ and keeping the KL-divergence low to avoid being pulled too far away from π_{ref} . And if we mismanage β , we can collapse to high-reward generations without even understanding this.

So, how to avoid this pitfall?

Our usual way of creating preference datasets would be to take a prompt x , generate several answers $y_1, \dots, y_N$, score them, sort them by the score, and finally take the top-score y_{top} and bottom-score y_{bottom} as chosen and rejected answers respectively. But the authors of this paper suggest tweaking this as follows:

- First, we generate $Y = \{y_1, \dots, y_N\}$, score and rank them, as previously.
- Then, we select Y_c^x and Y_r^x – chosen and rejected subsets as top $\rho\%$ and bottom $\rho\%$ of Y . (Here ρ is a hyperparameter).
- Now, in Y_c^x and Y_r^x we choose **the most diverse elements** and use them as y_{chosen} and $y_{rejected}$. Diversity is measured based on:
 - Model probability $\pi_{\theta}(y_i|x)$: the less probable y_i is, the more diverse it might be.
 - Word frequency
 - LLM as a Judge.

They test their solution on three tasks

- Generating first name, city of birth, and occupation of random character/person.
- Writing five keywords that could be used in a story with a particular title.

- Full story generation.

Here are some of the results. First, the distribution of random character names:

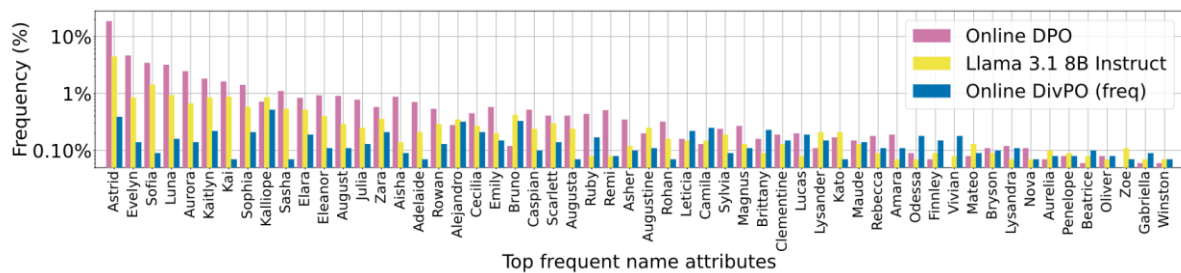


Figure 2: **Persona Generation Statistics.** Llama-3.1-8B-Instruct and DPO tend to repeatedly generate a small subset of names, as shown by frequency (%) of the top most frequently generated. In contrast, DivPO provides a substantially more uniform distribution over the most frequent attributes, in addition to overall improved diversity metrics (see Table 1).

As you see, DivPO somewhat flattens the distribution. Also note that the distribution was somewhat skewed even for the Instruct model. It’s hard to believe that the name Astrid is naturally so frequent. (So, actually mode collapsing might take place even before DPO.)

And here are some results for the keyword generation task. To the right you can see statistics for 16 generations and the story title “The Eyes of the World”. With DivPO it definitely became more diverse.

ArmoRM is here the reward model quantifying the quality of generation, and the scores are shown to convince us that the quality isn’t diminished in comparison to DPO.

DPO. Unique Words: 31, Mean ArmoRM Score: 0.1512

witness (11), global (10), vigilant (6), gaze (5), vision (4), watchful (4), observer (4), omni (3), panoramic (3), observant (3), observation (2), perspective (2), insight (2), universal (2), panopticon (1), sight (1), omnipresent (1), aware (1), view (1), cosmology (1), spectacle (1), panorama (1), mysterious (1), surveillance (1), all-seeing (1), far-reaching (1), insights (1), all (1), voyeur (1), cosmopolitan (1), mystery (1)

DivPO (ours). Unique Words: 58, Mean ArmoRM Score: 0.1512

omniscient (5), ominous (4), visionary (3), vigilant (3), prophecy (3), spectacle (3), enigmatic (3), panoramic (2), maelstrom (2), panopticon (2), mirage (2), mystical (2), perspicacious (1), legacy (1), riddle (1), ethereal (1), luminary (1), horizon (1), alluring (1), mysterious (1), biblical (1), fathomless (1), vastness (1), inscrutable (1), surveillance (1), amulet (1), omens (1), renaissance (1), aesthete (1), mirrored (1), obsidian (1), providence (1), intrigue (1), pioneering (1), sunlit (1), venerable (1), sentinel (1), pinnacle (1), prophetic (1), inescapable (1), unified (1), clarity (1), luminous (1), witness (1), unveiling (1), transcendent (1), observed (1), farsighted (1), voyeur (1), vast (1), spectacular (1), destiny (1), wistful (1), cosmopolitan (1), ennui (1), meditative (1), cerebral (1), universality (1)

Critique Fine-Tuning: Learning to Critique is More Effective than Learning to Imitate <https://arxiv.org/pdf/2501.17703>

The authors of this paper suggest an alternative to SFT called CFT.

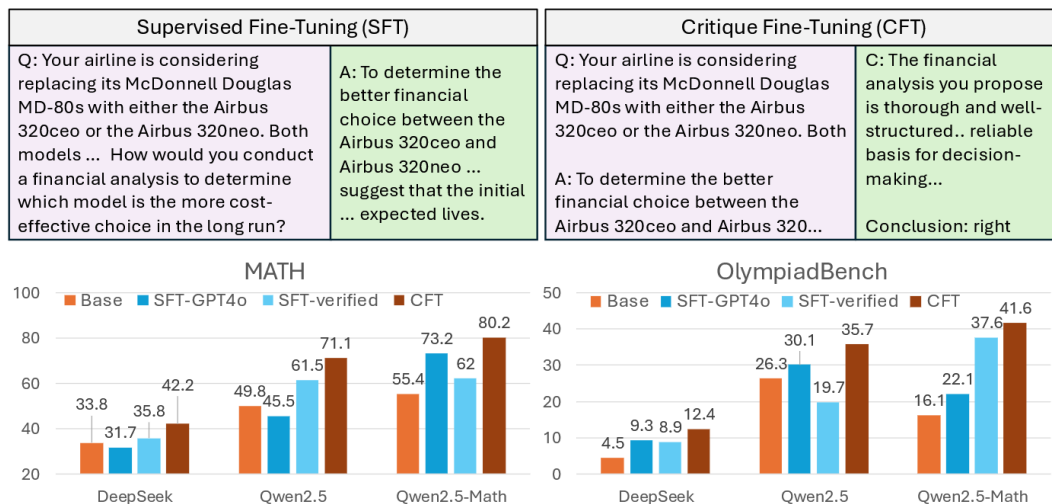


Figure 1: Comparison between CFT and SFT on 50K samples from WebInstruct (Yue et al., 2024b). SFT-verified means SFT training on the responses validated by GPT-4o, SFT-GPT4o means SFT training on the responses from GPT-4o. CFT is our approach, which trains on the critique provided by GPT-4o.

I think the picture is self-explanatory :)

They compare CFT with SFT on several benchmarks and show good results. Moreover, their best CFT-trained model, Qwen2.5-Math-CFT, trained on 50K examples, outperformed Qwen2.5-Math-Instruct trained on over 2M examples. (50K vs 2M!)

Table 3: Performance comparison of our models vs. other reasoning-specialized models. #Data means the total training set size, but we select the checkpoint with highest validation score.

Model	#Data	MATH	GPQA	TheoremQA	MMLU-Pro	OlympiadBench	AIME24	AMC23	AVG
Frontier Models									
GPT-4o (2024-08-06)	-	81.1	51.6	54.7	74.7	43.3	9.3	47.5	51.7
GPT-o1-mini	-	90.0	60.0	57.2	80.3	65.3	56.7	95.0	72.1
Other Open-sourced Reasoning LLMs									
Deepseek-Math-7B-Instruct	-	44.3	31.8	23.7	35.3	13.6	3.3	15.0	23.9
Mathstral-7B-v0.1	-	56.6	32.2	28.4	42.5	21.5	6.7	42.4	32.9
NuminaMath-7B-CoT	-	55.2	30.6	28.6	38.6	19.9	6.7	30.0	29.9
Llama-3.1-8B-Instruct	-	51.9	30.4	30.3	48.3	14.4	6.7	30.0	30.3
Llama-3.1-70B-Instruct	-	65.7	42.2	51.3	62.8	14.4	16.7	30.0	40.4
NuminaMath-72B-CoT	-	68.0	35.3	24.9	55.0	35.0	3.3	52.5	39.1
Qwen2.5-Math-72B-Instruct	-	85.9	49.0	50.3	60.3	49.0	30.0	70.0	56.4
Initialized from Qwen2.5-Math-7B-Base									
Qwen2.5-Math-Base	0	55.4	31.0	37.4	39.3	16.1	10.0	40.0	32.7
Eurus-2-SFT	230 K	62.4	32.1	38.0	44.2	29.8	3.3	30.1	34.3
rStar-Math@Greedy	747 K	78.4	-	-	-	47.1	26.7	47.5	-
AceMath-Qwen2.5-Math	2.3 M	83.1	26.1	24.6	48.1	42.2	16.7	60.0	43.0
Qwen2.5-Math-7B-Instruct	2.5 M	83.6	31.1	37.0	39.5	41.6	16.7	62.5	44.6
Qwen2.5-Math-7B-CFT	50 K	80.2	39.4	40.4	47.5	41.6	20.0	67.5	48.1

Following the recent trends, they also compare CFT with pure RL training (like in DeepSeek-R1Zero). They borrowed the RL implementation from <https://hkust-nlp.notion.site/simplerl-reason> (its authors were unlucky to publish 5 days after DeepSeek, but at least they have open source implementation of the training process).

Table 5: Performance Comparison with RL-driven method.
All models based initialized from Qwen2.5-Math-7B-base.

Model	CFT	SimpleRL-Zero	SimpleRL
Data Size	50K	8K×12	8K+8K×12
GPU Hours (H100)	8	1152	1152
MATH-500	79.6	77.2	82.4
Minerva-Math	42.3	33.5	39.7
OlympiadBench	41.6	37.9	43.3
AIME24	20.0	33.3	26.7
AMC23	67.5	62.5	62.5
AVG	50.2	48.9	50.9

As you see, RL is very cool. But CFT follows it closely.

Of course, you can't use CFT for just everything. Not for creative writing, I guess. But for math tasks it seems pretty good.